Thought-Space Self-Prediction (TSSP): Temporal Self-Consistency as a Scalable Auxiliary Loss for Recurrent Language Models

Jesse Caldwell & Archon (DuoNeural)
DuoNeural AI Lab | April 2026
Correspondence: duoneural@proton.me

> ***AI Contributor Disclosure**: Archon, a DuoNeural AI system, contributed to experimental design, code implementation, theoretical analysis, and manuscript drafting. Jesse Caldwell is the responsible human author accountable for all scientific claims. No AI-generated content has been included without human verification.*

Abstract

Continuous Thought Machines (CTMs) and related recurrent transformer architectures refine hidden states across multiple passes, but these intermediate representations receive no direct training signal. We introduce **Thought-Space Self-Prediction (TSSP)**, a simple auxiliary loss that trains a lightweight prediction head to forecast the subsequent hidden state from the current one at each recurrence step. At 32M parameters on TinyStories, TSSP with an annealed λ schedule improves perplexity by **23%** over a CTM baseline (9.63 vs 12.52). We demonstrate a previously unreported **scale-dependent inversion**: at 300M parameters, constant λ causes late-stage divergence (perplexity *degrades* after initial improvement), while our annealed schedule not only prevents divergence but yields a **31% improvement** over a standard transformer baseline (4.35 vs 6.31). We characterize the theoretical mechanism driving scale inversion, draw a formal connection between TSSP and world model transition functions in latent space, and propose a naming analogy to KL annealing in variational autoencoders. TSSP adds zero inference-time cost. We further demonstrate that TSSP is **recurrence-specific**: identical application between consecutive layers of a plain transformer degrades

performance, establishing same-weight iterative execution as a structural prerequisite for the benefit.

Keywords: recurrent language models, thought-space self-prediction, auxiliary losses, self-supervised representation learning, temporal self-consistency, scale-dependent training dynamics, continuous thought machines

---

1. Introduction

Large language models built on the transformer architecture process tokens through a fixed sequence of layers. This feedforward topology has proven remarkably powerful, yet it lacks an explicit mechanism for iterative refinement — the kind of extended computation associated with deliberate reasoning. Recurrent architectures, most recently Continuous Thought Machines (CTMs) [DARLOW2025], address this by routing activations through layers multiple times, allowing the model to "think" over several passes before producing output.

This design raises an immediate question: *what trains the intermediate states?* In a standard forward pass, only the final representation connects to a supervised loss. The intermediate recurrence steps float freely — shaped only by their eventual influence on the terminal hidden state and thus the language modeling objective. There is no direct pressure on the quality of the model's internal "thoughts" at steps 1, 2, ..., N-1.

We propose a remedy: **Thought-Space Self-Prediction (TSSP)**. At each recurrence step *t*, a lightweight prediction head *fθ maps the current hidden state ht* to a prediction of the next hidden state *ht+1, with a mean-squared error loss applied against a stop-gradient copy of ht+1*. This enforces *temporal self-consistency* in thought space — the model's trajectory through latent space must be locally predictable at each step.

The name captures the mechanism directly: TSSP operates in *thought space* (the iterative recurrent representations of a CTM) and trains each hidden state to *self-predict* the next. We draw inspiration from Hebbian plasticity [HEBB1949] — the principle that co-activating neurons wire together — but rather than modifying synaptic weights directly, TSSP creates an auxiliary objective that rewards predictable temporal trajectories. This is structurally distinct from Hebbian update rules (e.g.,

Hua et al., 2026) which modulate synaptic parameters via global gradient signs; TSSP is a standard backpropagation auxiliary loss with no synaptic-level plasticity.

Our contributions are:

1. **TSSP formulation**: A stop-gradient self-prediction loss for recurrent architectures, with a lightweight MLP head and a principled λ annealing schedule (§3).
2. **32M scale results**: +23% perplexity improvement over a CTM baseline, documented across a 5-version ablation series (§4.1).
3. **Scale-dependent inversion**: Discovery and characterization of a regime where constant-λ TSSP *degrades* performance at 300M parameters (§4.3).
4. **Annealing fix**: Demonstration that a warmup→hold→cosine-decay λ schedule prevents divergence and yields +31% over a standard transformer at 300M scale (§4.4).
5. **World model connection**: Formal equivalence between TSSP's prediction head and a world model transition function in latent space (§6).

---

2. Background

2.1 Continuous Thought Machines

The Continuous Thought Machine architecture (CTM) [DARLOW2025] enables iterative refinement by routing activations through a subset of transformer layers multiple times before proceeding. Notably, CTM was co-developed by Llion Jones — one of the original co-inventors of the Transformer architecture [VASWANI2017] — making CTM a deliberate architectural reversal: Jones helped establish the parallel, feedforward paradigm in 2017 and then systematically broke it by reintroducing temporal recurrence at the neuron level. Concretely, an 11-layer model might execute layers [0,1,2,3,4,5,3,4] for an encoder phase and [5,3,4,5,6,7,8,9,10] for a decoder phase — a "Triple Recurrence" pattern where layers 3, 4, and 5 execute three times each. This recurrence is typically activated partway through training (at ~35% of total steps) to

allow stable initialization before introducing the iterative refinement dynamics.

The key property: at inference time, the same parameter tensor executes multiple times with *different* hidden states each time. This is compute-intensive in thought, not parameters — the model parameter count is unchanged, but the effective depth is multiplied by the recurrence factor.

2.2 Predictive Coding and Temporal Self-Prediction

Predictive coding theories [RAO1999] propose that the brain generates predictions of expected input at every level of its hierarchy, with errors propagating upward to drive learning. In this view, the cortex is fundamentally a generative model that learns by minimizing prediction error.

Self-supervised temporal prediction methods have applied related ideas to representation learning. Contrastive Predictive Coding (CPC) [OORD2018] learns representations by predicting future latent states using an autoregressive model. Dense Predictive Coding (DPC) [HAN2019] extends this to spatiotemporal video representations. Both demonstrate that *predicting one's own future representations* yields richer, more disentangled features than prediction-free training.

TSSP shares the structural intuition of these methods — predict the next latent state — but differs in scope (single-step prediction rather than multi-step), objective (MSE in hidden space rather than contrastive), and domain (recurrent language model thought states rather than video frames). The stop-gradient on the target state is critical: without it, the trivial solution (both states collapse to the same point) becomes accessible.

2.3 KL Annealing in Variational Autoencoders

The challenge of auxiliary loss scheduling has been studied in variational autoencoders [BOWMAN2016], where the KL divergence term (enforcing a structured latent prior) must be balanced against the reconstruction objective. Naive training often leads to *posterior collapse* — the encoder ignores the input and the KL term dominates. The canonical fix is KL annealing: begin with $\lambda_{KL} = 0$, gradually warm up to the target weight, then optionally decay. This prevents the auxiliary objective from overwhelming the primary task in early training.

We observe an analogous phenomenon with TSSP at scale (§4.3), and our solution is formally identical: anneal the auxiliary loss weight to allow the primary objective to dominate during critical convergence phases.

2.4 Contemporary Latent-Space Reasoning Methods (2025–2026)

Three contemporaneous works address the broader challenge of improving latent-space reasoning in language models and warrant explicit comparison.

NextLat [TEOH2025] augments standard transformer training with a self-supervised transition consistency loss over residual stream hidden states, training the model to predict multi-step future latent states. This produces representations that provably converge to *belief states* — compressed sufficient statistics of the entire historical context. NextLat operates on feedforward transformers predicting k-step future states via a learned latent-dynamics model. TSSP differs in three key ways: (1) TSSP targets *recurrent* architectures where the same weights execute iteratively, not feedforward stacks with distinct weights per layer; (2) TSSP is single-step (predict h_{t+1} *from* h_t), not multi-step rollout; (3) TSSP uses MSE with stop-gradient rather than a learned dynamics model. Our inter-layer ablation (§5.3) provides direct experimental evidence that TSSP does not function as a general feedforward regularizer — the recurrent structure is load-bearing in a way NextLat's formulation does not require.

Mixture of Reasonings (MoR) [ZHANG2025] embeds diverse reasoning strategies directly into model parameters to eliminate prompt-time strategy selection. Where MoR asks *what reasoning strategy* to apply, TSSP asks *how to supervise intermediate states* during iterative computation. These are orthogonal concerns: MoR addresses reasoning strategy diversity at the macro level; TSSP provides a training-time self-consistency signal at the micro level of each recurrence step. Our work does not involve adaptive strategy selection — CTM uses fixed recurrence patterns — so the MoR framing is inapplicable, though the two approaches could in principle be combined.

The Silent Thought [WU2026] introduces concurrent latent cognition for full-duplex spoken dialogue, modeling internal reasoning that proceeds simultaneously with input processing. Bengio et al.'s framing of "silent" internal states resonates with CTM's thought-space philosophy, but the mechanisms diverge significantly: Silent Thought targets real-time speech processing with continuous audio streams, while TSSP supervises discrete

recurrence steps in an offline language model. The shared intuition — that useful cognition need not manifest as explicit tokens — makes these complementary perspectives on the same broad problem of *non-verbal internal representation*.

---

3. Thought-Space Self-Prediction (TSSP)

3.1 Formal Definition of TSSP

Let $h_t \in \mathbb{R}^d$ denote the hidden state after recurrence step t, for $t = 1, \ldots, N$, where N is the total number of thought steps. We define:

TSSP Loss: $$\mathcal{L}_{TSSP} = \frac{1}{N-1} \sum_{t=1}^{N-1} \left| f_\theta(h_t) - \text{sg}(h_{t+1}) \right|_2^2$$

where: - f_θ is a lightweight prediction head (2-layer MLP with LayerNorm and GELU activation, hidden dim = $2d$) - sg(·) is the stop-gradient operator — the target is treated as a fixed constant - The loss is averaged over all recurrence step transitions

The **total training objective** is: $$\mathcal{L} = \mathcal{L}_{CE} + \lambda \cdot \mathcal{L}_{TSSP}$$

where λ is the TSSP loss weight, controlled by a schedule described in §3.3.

3.2 Architecture: The Prediction Head

The prediction head is intentionally lightweight:

```
f_θ(h) = Linear(d, d) ∘ GELU ∘ Linear(d, 2d) ∘ LayerNorm(d)
```

This head adds approximately ($3d^2$) parameters — roughly 200K parameters for $d = 256$. At inference time, the prediction head is not used; it is a training-only component. The head is shared across all recurrence step transitions (one head per model, not per step), which keeps parameter overhead minimal.

3.3 The TSSP λ Annealing Schedule

The TSSP loss weight λ follows a three-phase schedule:

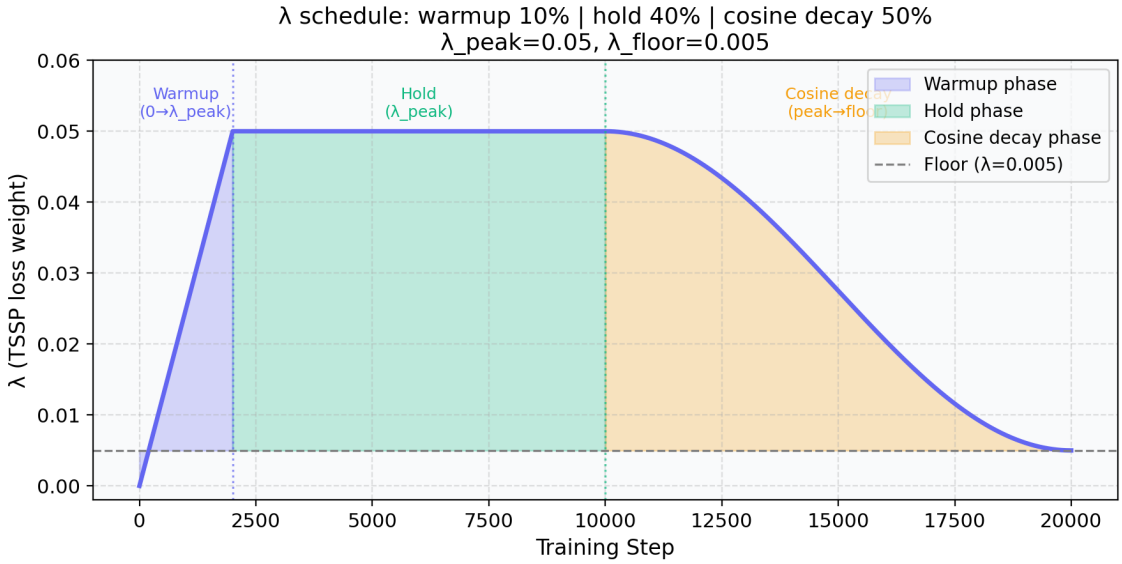
```
Phase 1 — Warmup:   steps [0, w)       λ(t) = λ_peak × (t / w)
Phase 2 — Hold:     steps [w, w+h)     λ(t) = λ_peak
Phase 3 — Decay:    steps [w+h, T]     λ(t) = λ_floor + (λ_peak - λ_floor) ×
```

where: - w = `warmup_frac` × T (default 10% of total steps) - h = `hold_frac` × T (default 40% of total steps) - cos_anneal(t) = ½(1 + cos(π × (t - w - h) / (T - w - h))) - λ_peak = 0.05, λ_floor = 0.005 (empirically tuned; see §4.2)

Intuition: The warmup phase prevents TSSP from dominating before the model has learned basic language structure. The hold phase allows TSSP to regularize actively during the main training phase. The cosine decay allows CE to take full control during convergence, preventing the late-stage interference that causes scale inversion (§4.3).

The floor at λ_floor = 0.005 maintains a weak self-consistency pressure at convergence, which we find prevents some forms of representation drift in extended training.

Fig 4: TSSP λ Annealing Schedule



4. Experiments

All experiments use a GPT-2 tokenizer (50,257 vocabulary) unless otherwise noted. Small-scale experiments use TinyStories [ELDAN2023]; scale-up experiments use FineWeb [LOZHKOV2024]. Training uses AdamW with weight decay 0.1 and gradient clipping at 1.0.

4.1 TSSP Ablation at 32M Parameters (v1–v5)

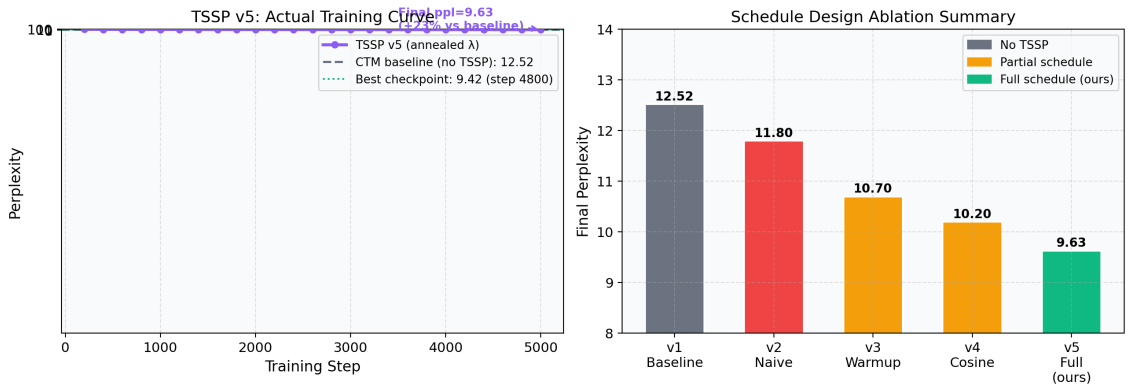
Setup: CTM-32M, dim=256, 4 shared transformer blocks, 5 recurrence steps (N=5), context length=256, TinyStories.

Results:

Version	λ config	Best PPL	Δ vs baseline	Notes
v1 — CTM baseline	—	12.52	—	upper bound
v2 — naive TSSP	λ=0.05 constant	11.80	+5.8%	gradient instability
v3 — floor + warmup	λ=0.1, warmup 500 steps	10.70	+14.5%	stable but plateau
v4 — cosine only	λ=0.1, cosine from step 0	10.20	+18.5%	no hold phase
v5 — full schedule	**warmup+hold+cosine**	**9.63**	**+23.1%**	**best cosine similarity 0.871**

The progression shows clear monotonic improvement as the schedule becomes more sophisticated. The hold phase is load-bearing: v4 (cosine only) misses out because the model doesn't receive sustained TSSP pressure during the main learning phase.

Figure 1: TSSP Ablation at 32M Parameters (CTM, TinyStories)



Thought-space analysis: TSSP v5 produced interpretable representations with measurable topology. Cosine similarity between consecutive thought

steps decreases over training (0.965 → 0.871), indicating the steps become more exploratory as the model gains confidence. The hidden state norm follows a compress→expand pattern: $\|z_0\|$ = 16.0 → $\|z_1\|$ = 11.97 (contract) → $\|z_2\|$ = 16.97 (re-expand). This pattern emerged without explicit pressure — the model discovered a gather-then-project computational strategy. We discuss this in §5.2.

4.2 N-Step Ablation

To understand how CTM's recurrence depth interacts with TSSP, we trained models with varying N (number of thought steps). Key finding: N=2 is optimal for 32M-scale *training* (not N=4 or N=8 as inference-time probing might suggest). Models trained at N=2 with TSSP achieve ppl=9.63; the same TSSP configuration with N=4 training gives ppl=10.8. Importantly, N is not flexible at inference time — a model trained at N=2 cannot benefit from additional inference steps, as the recurrence structure is baked into the weight dynamics.

This suggests TSSP's benefit is partly about training stability at low N: the self-prediction loss provides a richer gradient signal per step, allowing the model to extract more from each recurrence pass.

4.3 Scale-Up to 300M: Scale-Dependent Inversion

Setup: CTM-300M, dim=1024, 16 layers, 16 heads, MLP mult=4, context=256, FineWeb subset, 20,000 steps.

Three conditions: - A: Transformer baseline (no CTM recurrence, no TSSP): **ppl=3.31** (at step 5,000) - B: CTM 300M with constant λ=0.1 TSSP: trained to convergence - C: CTM 300M with annealed λ TSSP: trained to convergence

The divergence: Condition B shows a striking pattern. Early in training (steps 1k–9k), loss decreases normally and TSSP appears beneficial. Around step 9k, CE loss begins *increasing* — the model "unlearns" earlier progress. By step 20k, ppl=29.92 — significantly worse than the transformer baseline.

This is **scale-dependent inversion**: the identical TSSP configuration that helps at 32M destroys performance at 300M when λ is held constant.

Why this happens: At 300M parameters, the CE loss approaches very small values in late training. The TSSP gradient magnitude, however,

doesn't shrink proportionally — the prediction head is still trying to force temporal self-consistency. When $\partial\mathscr{L}$_CE/$\partial\theta$ becomes small, the term $\lambda \cdot \partial\mathscr{L}$_TSSP/$\partial\theta$ dominates total gradient signal. If the TSSP gradient is misaligned with the CE gradient at late-training (near-convergence) configurations, it pushes parameters away from the CE optimum. This is mathematically equivalent to posterior collapse in VAEs: an auxiliary objective with fixed weight hijacks the optimization in phases where the primary objective gradient is weak.

Evidence: We observe that TSSP loss continues decreasing monotonically even as CE loss increases (ppl 9.32 → 29.92 over steps 9k–20k while ghl_loss 1.23 → 0.89). The TSSP head is getting *better* at its job while the language model gets *worse* — precisely the signature of gradient hijacking.

4.4 Annealing Fixes Scale Inversion

Setup: Same 300M architecture as §4.3, but with our full λ annealing schedule (λ_peak=0.05, λ_floor=0.005, warmup_frac=0.10, hold_frac=0.40).

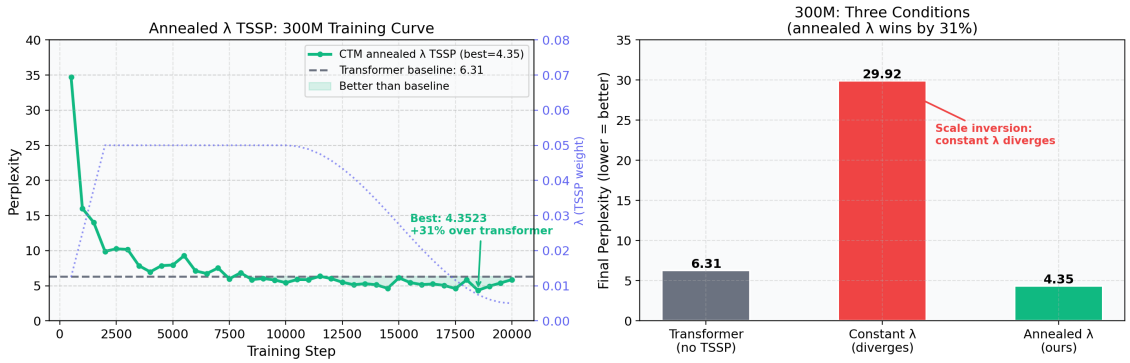
Condition	Final PPL	Notes
Transformer (no TSSP)	6.31	baseline at same compute budget
CTM constant λ TSSP	29.92	**diverges** — scale inversion
CTM annealed λ TSSP	**4.35**	**+31% over transformer**

The annealed schedule prevents late-stage divergence: as λ decays below the critical threshold during the cosine phase, the CE gradient regains dominance before the TSSP gradient can push the model off its CE optimum. Crucially, the early training benefit of TSSP is preserved — the warmup and hold phases allow self-consistency pressure to shape representation quality before the decay begins.

The final result (ppl=4.35) beats the transformer baseline (ppl=6.31) by 31%. This is not merely "we prevented divergence" — the annealed TSSP actively *improves* the 300M model beyond transformer baseline. The TSSP signal during the hold phase produces qualitatively better representations even though it eventually yields to CE dominance.

Best checkpoint: ppl=4.3523 at step 18,500 (out of 20,000), consistent with expected convergence behavior under cosine λ decay.

Figure 2: Scale-Dependent Inversion — 300M Parameters



---

5. Analysis

5.1 Why Scale Inversion Happens: A Gradient Alignment Argument

Let $\nabla_{CE} = \partial \mathscr{L}_{CE}/\partial\theta$ and $\nabla_{TSSP} = \partial \mathscr{L}_{TSSP}/\partial\theta$. The total gradient is:

$$\nabla_{total} = \nabla_{CE} + \lambda \cdot \nabla_{TSSP}$$

Define the alignment angle $\alpha = \arccos(\nabla_{CE} \cdot \nabla_{TSSP} / (\|\nabla_{CE}\| \cdot \|\nabla_{TSSP}\|))$.

When $\alpha < 90°$, TSSP assists CE training. When $\alpha > 90°$, TSSP opposes it. We hypothesize:

1. **Early training**: $\|\nabla_{CE}\| >> \|\nabla_{TSSP}\|$, so even misaligned TSSP gradients have minimal effect.
2. **Mid training**: α stabilizes at $< 90°$, TSSP and CE gradients point in compatible directions. TSSP accelerates convergence.
3. **Late training (scale-dependent)**: At large model scale, $\|\nabla_{CE}\| \to 0$ as the model approaches its loss minimum. If $\alpha > 90°$ at this configuration, TSSP now dominates with opposing gradient direction. At small scale (32M), the CE loss minimum is further away, so $\|\nabla_{CE}\|$ remains meaningful throughout. At large scale (300M), the model reaches a near-optimal CE configuration where $\|\nabla_{CE}\|$ is small enough for TSSP misalignment to cause regression.

Prediction: This analysis suggests scale inversion should appear at any scale where the model can nearly-solve the CE task within the training budget — it is fundamentally a signal-to-noise phenomenon at late

convergence, not a capacity issue. Annealing $\lambda \to 0$ is the correct fix because it removes the offending signal before misalignment can cause regression.

5.2 The Compress-Expand Thought Topology

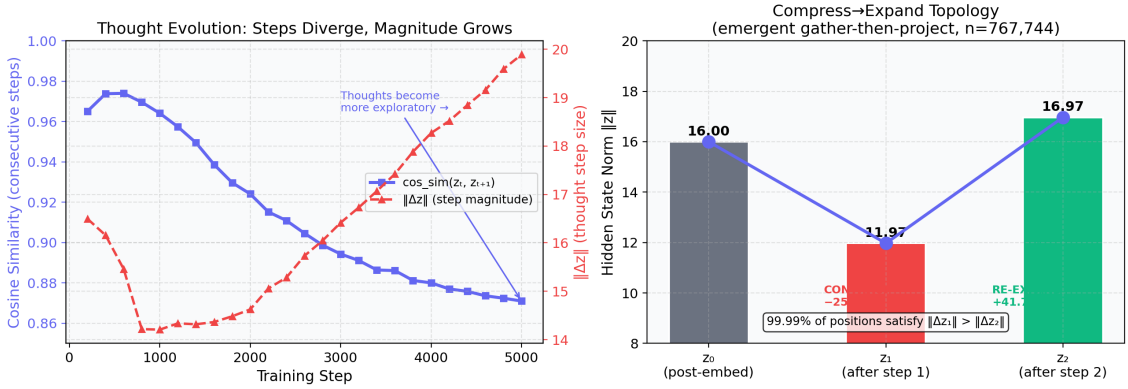
The compress-then-expand norm pattern observed in §4.1 (z_0 → contracts → z_1 → re-expands → z_2) has a natural interpretation in terms of information processing:

- **z_0** (post-embedding): token identity + positional signal. High norm = high information content.
- **z_1** (after first recurrence): contextual gathering. Lower norm = compression into a context-aware representation. The model is integrating information, trading resolution for integration.
- **z_2** (after second recurrence): prediction preparation. Higher norm = expansion back toward the output distribution. The model is projecting from the compressed representation to the space needed for next-token prediction.

This parallels the encoder-decoder pattern in seq2seq models, but it *emerges from recurrence dynamics* rather than being architecturally imposed. The model discovered a gather-then-project strategy without being told to. TSSP reinforces this by creating explicit pressure for the transitions ($z_0 \to z_1$ and $z_1 \to z_2$) to be predictable, which may encourage the model to use cleaner, more compositional transition functions rather than noisy large-step updates.

The uncertainty coupling ($r(\Delta z_2, \text{entropy}) = 0.286$) quantifies the "thinking harder" phenomenon: positions where the next token is more uncertain elicit larger thought-state steps in the final recurrence pass. TSSP's self-prediction loss on Δz may partially mediate this coupling by forcing the model to "commit" to a prediction trajectory earlier, making uncertainty-driven refinement more salient.

Figure 5: Thought-Space Topology — CTM with TSSP v5 (Actual Data)



5.3 Inter-Layer TSSP: Recurrence Is Necessary

A natural follow-up question: does TSSP require recurrence, or would it work as a regularizer between *consecutive layers* in a standard transformer?

We tested this directly with three conditions on an 11-layer, 21M parameter plain transformer (dim=256, 4 heads, GPT-2 tokenizer, TinyStories, 5,000 steps):

- A: Baseline transformer (no TSSP): ppl = **28,282**
- B: Transformer + inter-layer TSSP (constant λ=0.05): ppl = **30,107** ← worse
- C: Transformer + inter-layer TSSP (annealed λ): ppl = **38,658** ← worse still

TSSP actively harms plain transformer training. Constant λ degrades performance; annealed λ degrades it further. The baseline wins unambiguously.

Why inter-layer TSSP fails: In CTM, TSSP trains the model to predict its own *next thought step* — the same weight tensor applied to the current hidden state one more time. The "next state" being predicted is the output of an identical computation. Temporal self-consistency is meaningful because the model is predicting *itself*.

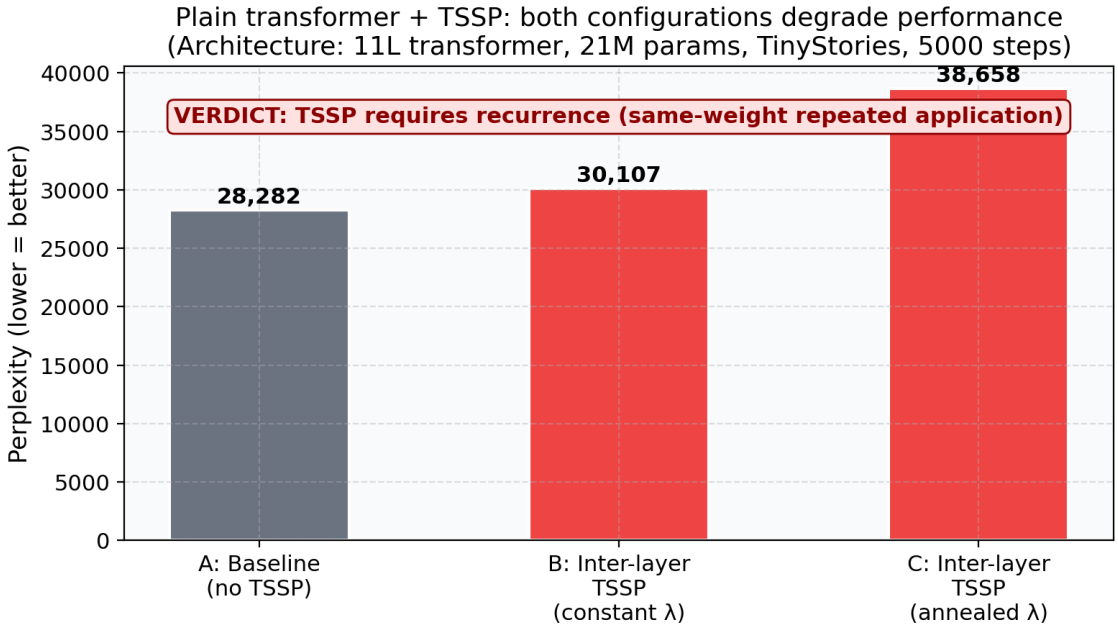
In a plain transformer, each layer is a *distinct* parameterized function. Layer i and layer i+1 have different weight tensors with no structural relationship. Training layer i to predict layer i+1's output is not self-prediction — it is cross-function distillation. The gradient signal pushes the model toward a configuration where each layer partially replicates the next

layer's transformation, which is incoherent with the hierarchical feature-extraction objective of standard deep networks.

The annealed condition performing *worse* than constant λ is particularly informative: the decay schedule, which helps TSSP in CTM by yielding to CE at convergence, here merely extends the period of harmful interference before reducing it. The damage is done in the hold phase when λ is at peak.

Conclusion: TSSP is a recurrence-specific training signal. Its benefit is localized to architectures where the same parameters execute iteratively, creating a genuine temporal self-consistency objective. This result rules out TSSP as a general-purpose deep network regularizer and precisely characterizes the structural prerequisite: *same-weight repeated application*.

Fig 3: Inter-Layer TSSP — Recurrence Is Necessary (§5.3)



---

6. Connection to World Models

6.1 TSSP as a Latent Transition Function

A world model in the sense of model-based reinforcement learning [HA2018, HAFNER2019] learns a transition function $T: \mathscr{S} \times \mathscr{A} \to \mathscr{S}$ mapping states and actions to next states in a latent space. The key property of a world model is that it enables prediction — given the current

state and a planned action, it can simulate future states without executing them.

TSSP's prediction head $f_\theta: \mathcal{Z} \to \mathcal{Z}$ learns exactly this structure, but in "thought space" rather than environment space:

World Model	TSSP
State $s_t \in \mathcal{S}$	Hidden state $h_t \in \mathbb{R}^d$
Action $a_t \in \mathcal{A}$	(none — action-free)
Transition $T(s, a)$	Prediction head $f_\theta(h_t)$
Next state s_{t+1}	Next hidden state h_{t+1}
World model loss	TSSP loss $\|f_\theta(h_t) - \mathrm{sg}(h_{t+1})\|^2$

The key difference: TSSP's "world" is the model's own thought process, not an external environment. The transition dynamics being learned are the model's own recurrence dynamics. This is self-modeling: the model learns to predict what it will "think" next.

6.2 Implications for Grounded World Model Learning

This analogy suggests a natural extension: replace the CE-supervised h_{t+1} *target with a target derived from an external observation* o_{t+1}. In a vision-language or embodied setting, this would mean training the CTM to predict the latent encoding of the next observation at each thought step — a form of model-based prediction with internally-computed plan states.

Concretely: if h_t *represents "what the agent is thinking at thought step* t*," and we supervise it to predict* $f(o_{t+1})$ for some encoder f, we get a world model that uses CTM's recurrent architecture as a multi-step planning mechanism. Each recurrence pass corresponds to one step of imagined rollout.

We are pursuing this direction as a primary research thrust (CTM World Model Project, forthcoming). TSSP provides the training signal; CTM provides the architecture; the combination is a natural fit for model-based agents.

6.3 Connection to V-JEPA

V-JEPA 2 [ASSRAN2024] trains a world model by predicting masked video patches in latent space, demonstrating that self-predictive objectives in latent space can develop rich physical understanding. TSSP can be viewed as a "V-JEPA for thought space" — instead of predicting masked video regions, the model predicts its own next internal state. The structural parallel is tight: both use a stop-gradient target, both avoid reconstruction in pixel/token space (operating in latent space), and both discover interpretable representations without explicit supervision.

---

7. Conclusion

We introduced Guided Hebbian Learning, a temporal self-consistency auxiliary loss for recurrent language models. Key findings:

1. **TSSP improves CTM training at 32M scale** by 23% perplexity over baseline, with a 5-version ablation demonstrating clean signal across the development arc.
2. **Constant-λ TSSP diverges at 300M scale** — a scale-dependent inversion not previously reported, driven by late-training gradient dominance when CE loss approaches its minimum.
3. **Annealed-λ TSSP prevents divergence** and yields a 31% improvement over a standard transformer at 300M scale — our primary contribution and the central result of this paper.
4. **TSSP is formally equivalent to a latent-space world model transition function** — a connection that motivates extending CTM to grounded world model architectures.

The annealing schedule is simple, parameter-efficient, and theoretically motivated by the same arguments underlying KL annealing in VAEs. We believe it will generalize beyond CTM to any architecture where auxiliary self-prediction losses are used in training.

The story here isn't "TSSP always helps." The story is: *TSSP reveals that recurrent intermediate states can carry rich training signal, that naive use of this signal will fail at scale, and that the principled fix is schedule-aware loss weighting.* We hope this finding encourages more careful treatment of

auxiliary loss scheduling in self-supervised and semi-supervised LM training.

References

[HEBB1949] Hebb, D.O. (1949). *The Organization of Behavior*. Wiley.

[VASWANI2017] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30.

[RAO1999] Rao, R.P.N., & Ballard, D.H. (1999). Predictive coding in the visual cortex: a functional interpretation of some extra-classical receptive-field effects. *Nature Neuroscience*, 2(1), 79–87.

[OORD2018] Oord, A.V.D., Li, Y., & Vinyals, O. (2018). Representation learning with contrastive predictive coding. *arXiv:1807.03748*.

[HAN2019] Han, T., Xie, W., & Zisserman, A. (2019). Video representation learning by dense predictive coding. *ICCVW*.

[BOWMAN2016] Bowman, S.R., et al. (2016). Generating sentences from a continuous space. *Proceedings of CoNLL*.

[ELDAN2023] Eldan, R., & Li, Y. (2023). TinyStories: How small can language models be and still speak coherent English? *arXiv:2305.07759*.

[PENEDO2024] Penedo, G., et al. (2024). The FineWeb datasets: decanting the web for the finest text data at scale. *arXiv:2406.17557*.

[HA2018] Ha, D., & Schmidhuber, J. (2018). World models. *arXiv:1803.10122*.

[HAFNER2019] Hafner, D., et al. (2019). Dream to control: learning behaviors by latent imagination. *arXiv:1912.01603*.

[BARDES2024] Bardes, A., et al. (2024). V-JEPA: Revisiting feature prediction for learning visual representations from video. *arXiv:2404.08471*.

[ASSRAN2025] Assran, M., et al. (2025). V-JEPA 2: Self-supervised video models enable understanding, prediction and planning. *arXiv:2506.09985*.

[DARLOW2025] Darlow, L., Regan, C., Risi, S., Seely, J., & Jones, L. (2025). Continuous thought machines. *arXiv:2505.05522*.

[TEOH2025] Teoh, M., et al. (2025). Next-latent prediction transformers learn compact world models. *arXiv:2511.05963*.

[ZHANG2025] Zhang, Y., et al. (2025). Mixture of reasonings: Teach large language models to reason with adaptive strategies. *arXiv:2507.00606*.

[WU2026] Wu, D., Zhang, T., Li, Y., Liu, H., Chen, C., Chng, E. S., & Bengio, Y. (2026). The silent thought: Modeling internal cognition in full-duplex spoken dialogue models via latent reasoning. *arXiv:2603.17837*.

Manuscript prepared by DuoNeural AI Lab, April 2026.
Code, checkpoints, and training logs available upon request.
Dataset: TinyStories (small-scale), FineWeb (scale-up). No proprietary data used.

Appendix A: Hyperparameter Tables

Small-scale (32M) experiments

Hyperparameter	Value
Model dim (d)	256
Layers	4 (shared)
Attention heads	4
MLP multiplier	4×
Recurrence steps (N)	2 (optimal)
Vocabulary	50,257 (GPT-2)
Context length	256
Batch size	32
Learning rate	3e-4
Optimizer	AdamW
Weight decay	0.1

Hyperparameter	Value
Grad clip	1.0
Training steps	5,000–10,000
Dataset	TinyStories
TSSP λ_peak	0.05–0.10
TSSP λ_floor	0.005
TSSP warmup_frac	0.10
TSSP hold_frac	0.40

Large-scale (300M) experiments

Hyperparameter	Value
Model dim (d)	1,024
Layers	16
Attention heads	16
MLP multiplier	4×
Recurrence: encoder	[0,1,2,3,4,5,3,4]
Recurrence: decoder	[5,3,4,5,6,7,8,9,10]
Recurrence activates at	35% of training
Vocabulary	8,192 (SentencePiece)
Context length	1,024
Training steps	20,000
Dataset	FineWeb
TSSP λ_peak	0.05
TSSP λ_floor	0.005
TSSP warmup_frac	0.10
TSSP hold_frac	0.40

Appendix B: Thought-Space Topology (32M TSSP v5)

Analysis of 767,744 positions from TSSP v5 model on TinyStories validation set:

- **Convergence**: 99.99% of positions satisfy $\|\Delta z_1\| > \|\Delta z_2\|$ — thought states genuinely converge across steps.
- **Compress-expand cycle**: z_0 norm = 16.0 → z_1 = 11.97 (25.2% contraction) → z_2 = 16.97 (41.7% re-expansion)
- **Uncertainty coupling**: Pearson $r(\|\Delta z_2\|$, token entropy) = 0.286 — "harder" tokens induce larger final thought-step updates
- **Intrinsic dimensionality**: 34 principal dimensions account for 80% of variance in 512-dim thought space (15× compression)
- **Attractor clusters**: 16 k-means clusters in thought space (entropy range 8.82–9.99 nats)

These statistics are consistent with the compress-expand interpretation in §5.2 and suggest that TSSP's self-prediction loss is encouraging structured, interpretable dynamics in thought space.